EECS 484, Winter 2026 - Database Management Systems

Download SQLite:
https://www.sqlite.org/download.html

The Relational Model

Chapter 3

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

Tea Time: Golden Monkey Tea

金猴茶

jīn hóu chá

- Origin: Yunnan province
- Second place in 2009 World Tea Championship
- Grown at 4000 ft altitude

ER Modeling: Summary

- High-level description of data to be stored
- ER model popular for conceptual design
  - Constructs are expressive and natural
  - Basic constructs: entities, relationships, and attributes (of entities and relationships)
  - Additional constructs: weak entities, ISA, aggregation
  - Integrity constraints: key constraints and participation constraints
- ER designing is subjective! Follow the requirements, state any other assumptions.
- Do not overuse the ISA hierarchy.

Relational (Data) Model

- **Schema** = a description of data in terms of a data model
- **Instance** = a table, with rows (aka tuples, records), and columns (aka fields, attributes) that match the schema
  - # of rows: cardinality
  - # of columns: degree or arity

Students

sid	name	login	age
13	Lisa	lsimp	40
41	Bart	bart	20

New Scenario: Olympic Games

- Some history:
  - Inspired by the ancient Olympic Games, which were held in Olympia, **Greece** (8th century BC).

Example: Instance of Athlete Relation

AID	Name	Country	Sport
1	Mary Lou Retton	USA	Gymnastics
2	Jackie Joyner-Kersee	USA	Track
3	Michael Phelps	USA	Swimming

What is the schema?

(aid: integer, name: `string`, country: `string`, sport: `string`)

Cardinality & Degree?

(A) Cardinality: 3, Degree: 3
(B) Cardinality: 3, Degree: 4
(C) Cardinality: 4, Degree: 3

Example: Instance of Athlete Relation

AID	Name	Country	Sport
1	Mary Lou Retton	USA	Gymnastics
2	Jackie Joyner-Kersee	USA	Track
3	Michael Phelps	USA	Swimming

What is the schema? (aid: integer, name: `string`, country: `string`, sport: `string`)

Cardinality & Degree? Cardinality = 3, Degree = 4

Relational Query Languages

- Supports simple, powerful querying of data
- Queries written declaratively
  - In contrast to imperative languages
- DBMS is responsible for efficient evaluation
  - System can optimize for efficient query execution, and still ensure that the answer does not change
- SQL is the standard database query language

Structured Query Language (SQL)

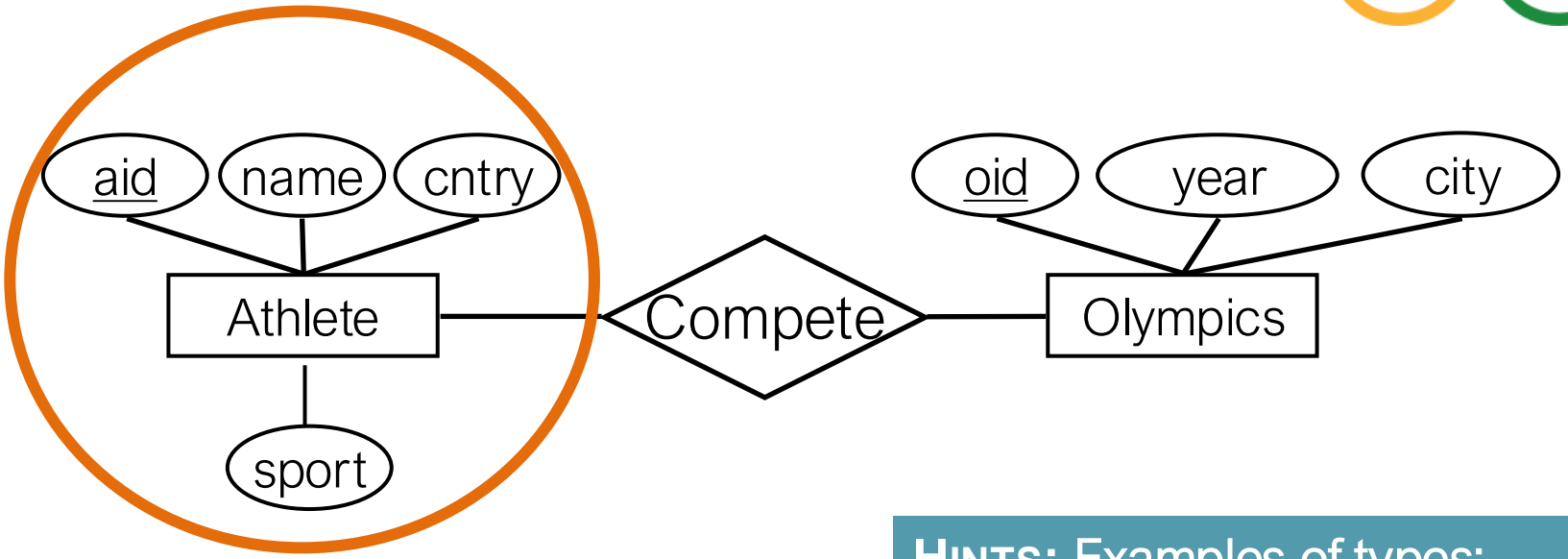
- Create a Table `Create`
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Create a Table (Relation)

```
CREATE TABLE table_name (
  field1 TYPE,
  field2 TYPE,
  … … …
);
```

Try it out on paper or SQLite



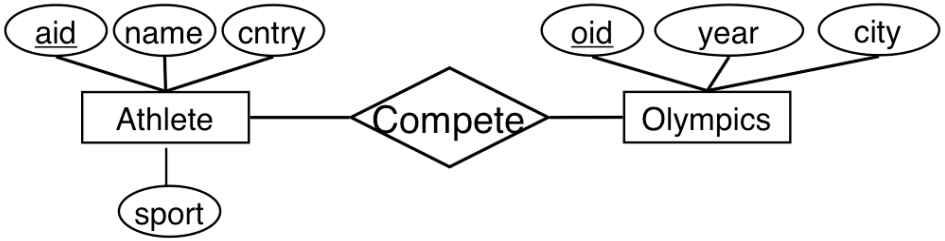

```
CREATE TABLE table_name (
   field1 TYPE,
   field2 TYPE,
    … … …
);
```

HINTS: Examples of types:

- INTEGER, REAL, NUMBER, …
- CHAR(20) = **always 20 chars**, padded with spaces
- VARCHAR(20) = **up to 20 characters**
- TEXT = **"large string"**, often no practical limit
- Blob = binary data (treated as raw data

Download SQLite:
https://www.sqlite.org/download.html

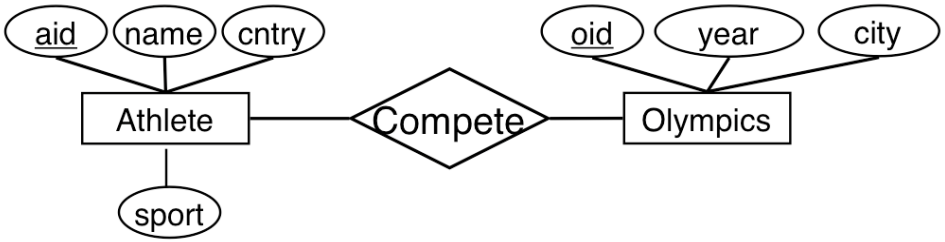
Creating Relations in SQL



- Create the Athlete relation
  - Domain constraint (type) enforced when tuples added or modified

```
CREATE TABLE Athlete
(aid INTEGER,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

Creating Relations in SQL



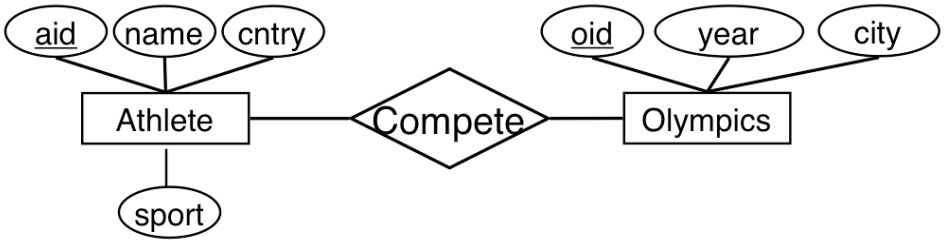
- Create the Athlete relation
  - Domain constraint (type) enforced when tuples added or modified

```
CREATE TABLE Athlete
(aid INTEGER,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

- Create the Olympics relation

```
CREATE TABLE Olympics
(oid INTEGER,
 year INTEGER,
 city CHAR(20));
```

Creating Relations in SQL



- Create the Athlete relation
  - Domain constraint (type) enforced when tuples added or modified

```
CREATE TABLE Athlete
(aid INTEGER,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

- Create the Olympics relation

```
CREATE TABLE Olympics
(oid INTEGER,
 year INTEGER,
 city CHAR(20));
```

- Create the Compete relation

```
CREATE TABLE Compete
(aid INTEGER,
 oid INTEGER);
```

Structured Query Language (SQL)

- Create a Table `Create`
  - Integrity Constraints
  - Enforcing Constraints
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Creating Relations: Constraints

- How to specify certain attributes as keys?
  - e.g., athlete ID (aid) or olympics ID (oid)
  - We must prevent duplicate keys, e.g. two athletes with the same ID in the database

- How to say that the Athlete ID and Olympic ID values in Compete relation must have valid references?

Integrity Constraints (ICs)

- IC: condition that must be true for *any* instance of the database; e.g., domain constraints
  - ICs are specified when schema is defined
  - ICs are checked when relations are modified
- A legal instance of a relation satisfies *all* specified ICs
  - DBMS must not allow illegal instances

Integrity Constraint: Primary and Candidate Keys

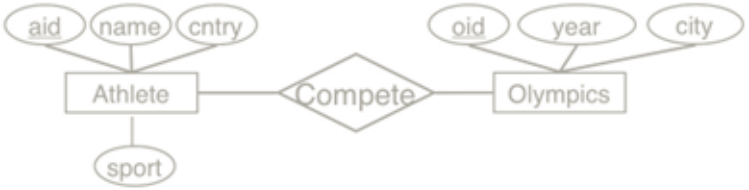
- A key for a relation R
  - = minimal set of attributes $A_1, \ldots, A_n$ such that:
  - no two tuples in (any instance of) R can have the same values for $A_1, \ldots, A_n$
- A relation can have more than one key:
  - One is designated as primary key.
  - Others are called candidate keys.

Examples: {aid}, {ssn}, {ssn, name}

- {aid} is a key in the Athlete relation
- {ssn} is a key for Citizen relation
- {ssn, name} is **not** a key, but a superkey – not minimal

PRIMARY KEY Constraint

Several ways of specifying the constraint:



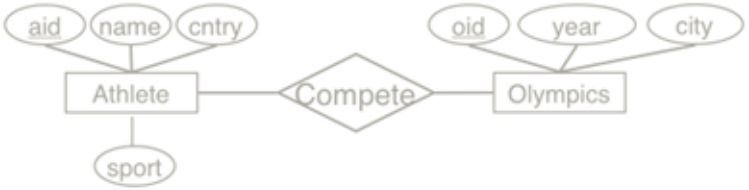

```
CREATE TABLE Athlete
(aid INTEGER PRIMARY KEY,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

```
CREATE TABLE Athlete
(aid INTEGER,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20),
 PRIMARY KEY(aid));
```

```
CREATE TABLE Compete
(aid INTEGER,
 oid INTEGER,
 PRIMARY KEY (aid, oid));
```

NOT NULL Constraint

Disallow null values for a field




```
CREATE TABLE Athlete
(aid INTEGER PRIMARY KEY,
 name CHAR(30) NOT NULL,
 country CHAR(20),
 sport CHAR(20));
```

- NULL value = the value is unknown or inapplicable
- Example:
  - country and sport can be NULL (= not known or unspecified)
  - But, name must be specified.

Primary Keys Properties

- Can **never** be null (databases automatically enforce this)
  - NO parts of a composite primary key can be NULL.
    - Note: Some DBMSs do not enforce this (e.g., sqlite)
- Need not be an integer ID
  - though they often are for efficient search
- IDs used as primary keys do not necessarily auto-increment in databases.
  - Additional features of SQL must be used to make them auto-increment. You will see that in the projects.

Candidate Keys

- Candidate keys specified using UNIQUE
- One of the candidate keys is chosen as the *primary key*.

```
CREATE TABLE Athlete
      (aid INTEGER,
       name CHAR(30) NOT NULL,
       country CHAR(20),
       sport CHAR(20),
      UNIQUE (name, country),
      PRIMARY KEY (aid));
```

In English, what restriction does the candidate key impose here?

- Note: NULL values do not count as duplicates
  - Can have both (1, 'Mike', null, 'Track') & (2, 'Mike', null, 'Swim')

WARNING: If used carelessly, ICs can prevent storing instances that arise in practice!

Foreign Keys in SQL

- Only people listed in Athletes relation should be allowed to compete

```
CREATE TABLE Compete
(aid INTEGER,  oid INTEGER,
 PRIMARY KEY (aid, oid),
 FOREIGN KEY (aid) REFERENCES Athlete (aid));
```

- … and only in games stored in the Olympics relation

```
CREATE TABLE Compete
(aid INTEGER,  oid INTEGER,
 PRIMARY KEY  (aid, oid),
 FOREIGN KEY (aid) REFERENCES Athlete (aid),
 FOREIGN KEY (oid) REFERENCES Olympics (oid));
```

Here, Athlete is called the parent/referenced table, and Compete is called the child/referencing table

Foreign Keys: Definition and Rules

- Foreign key = set of fields in one relation that is used to refer to a **sinlge** tuple in another relation.
- Must refer to a candidate key of the parent relation (can be a primary key or unique)
  - Like a 'logical pointer'
- Example:

```
CREATE TABLE Athlete
(aid INTEGER PRIMARY KEY,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

```
CREATE TABLE Compete
    (aid INTEGER,  oid INTEGER,
      PRIMARY KEY  (aid, oid),
      FOREIGN KEY (aid) REFERENCES Athlete (aid),
      FOREIGN KEY (oid) REFERENCES Olympics (oid));
```

If all foreign key constraints are enforced, referential integrity (no dangling references) is achieved.

Sample Database

- On Canvas: athlete_create.sql
  - Contains commands to create a database with these three relations
  - and some data
- You can load it into SQLite as follows:

```
% sqlite3 athlete.db
.read athlete_create.sql
```

- If you use sqlplus, use the following:

```
START athlete_create.sql
```

Structured Query Language (SQL)

- Create a Table `Create`
  - Integrity Constraints
  - Enforcing Constraints
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Enforcing ICs

- Whenever we modify the database
  - must check for violations of ICs
- Enforcing Domain, Primary Key, Unique ICs is straightforward
  - Reject offending UPDATE / INSERT command

Enforcing Referential Integrity

- If a Compete tuple is inserted with no corresponding Athlete aid:
  - Insert operation is REJECTED!

```
CREATE TABLE Compete
   (aid INTEGER,  oid INTEGER,
    PRIMARY KEY  (aid, oid),
    FOREIGN KEY (aid) REFERENCES Athlete (aid),
    FOREIGN KEY (oid) REFERENCES Olympics (oid)
   );
```

Enforcing Referential Integrity

- If a Compete tuple is inserted with no corresponding Athlete aid:
  - Insert operation is REJECTED!
- What if an Athlete tuple is deleted? Possible actions:
  - **Disallow deletion** if a Compete tuple refers to athlete
  - **Delete all** Compete tuples that refer to deleted athlete
  - **Set to default or null** value for all references to the deleted athlete
- Similar choices on update of primary key of Athlete

Referential Integrity in SQL

- When parent key is updated/deleted:
  - NO ACTION: no action taken (similar to RESTRICT but the check is deferred till commit)
  - RESTRICT: update/delete of parent key is disallowed when there's a ref to it in the child
  - CASCADE: also delete/update all tuples that refer to deleted tuple
  - SET NULL / SET DEFAULT: sets foreign key value of referencing tuple

```
CREATE TABLE Compete
    (aid INTEGER,  oid INTEGER,
      PRIMARY KEY  (aid, oid),
      FOREIGN KEY (aid)
      REFERENCES Athlete (aid)
      ON DELETE CASCADE
      ON UPDATE NO ACTION)
```

What happens if we modify an athlete's ID with an associated Compete tuple?

*** Warning:** On some DBMSs RESTRICT & NO ACTION are the same

Try it out

- Modify athlete_create.sql so that it has the UPDATE and DELETE constraints in COMPETE relation as in the previous slide.
  - Modified file available in athlete_modified.sql.
- Try the following and check COMPETE:

```
sqlite> DELETE FROM Athlete WHERE name='Michael Phelps';
sqlite> UPDATE Athlete SET aid=5 WHERE aid=4;
```

Implementation Note: In SQLite, make sure you issued "`PRAGMA foreign_keys = ON;`" command to enforce foreign key constraints. By default, SQLite ignores them for backward compatibility.

Implementation Notes

- Oracle's sqlplus:
  - You cannot use NO ACTION constraints. They are the default and thus not needed.
  - String literals like 'USA' must use **single quotes**, not double quotes

- SQLite:
  - You need `PRAGMA foreign_keys = ON;` to enforce foreign key constraints. This is for backward compatibility.

Where do ICs Come From?

- Based on real-world enterprise being modeled
- An IC is a statement about all possible instances!
- We can **check** a database instance to see if an IC is **violated**, but we can NEVER infer that an IC is true by looking at an instance.
- Key and foreign key ICs are the most common
- Also table constraints and assertions (Next week!)

Destroying & Altering Relations

- To destroy the relation Olympics.
  - Schema information and tuples are deleted

```
DROP TABLE Olympics
```

- To alter the Athlete schema by adding a new column

```
ALTER TABLE Athlete
    ADD COLUMN age: INTEGER
```

- What do we put in the new field?
- A null value: 'unknown' or 'inapplicable'

Structured Query Language (SQL)

- Create a Table `Create`
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Adding & Deleting Tuples

- Insert a single tuple using:

```
INSERT INTO  Athlete (aid, name, country, sport)
VALUES  (4, 'Johann Koss', 'Norway', 'Speedskating')
```

```
INSERT INTO  Athlete (country, name, sport, aid)
VALUES  ('Norway', 'Johann Koss', 'Speedskating', 4)
```

- Delete all tuples satisfying some condition (e.g. Smith)

```
DELETE
FROM Athlete A
WHERE A.name = 'Smith' AND A.sport = 'Track'
```

Structured Query Language (SQL)

- Create a Table `Create`
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Update Statement

- What does the following command do?

```
UPDATE Athletes A
SET A.country = 'United States'
WHERE A.country = 'USA' AND A.aid <= 2;
```

Athletes:

AID	Name	Country	Sport
1	Mary Lou Retton	USA	Gymnastics
2	Jackie Joyner-Kersee	USA	Track
3	Michael Phelps	United States	Swimming

Structured Query Language (SQL)

- Create a Table `Create`
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Select Statement

- What does this **query** compute?

```
SELECT name
FROM Athletes
WHERE country = 'USA';
```

Athletes:

AID	Name	Country	Sport
1	Mary Lou Retton	USA	Gymnastics
2	Jackie Joyner-Kersee	UK	Track
3	Michael Phelps	USA	Swimming

Select Statement

- What does this **query** compute?

```
SELECT name
FROM Athletes
WHERE country = 'USA';
```

"Find the names of all athletes from 'USA'

Athletes:

AID	Name	Country	Sport
1	Mary Lou Retton	USA	Gymnastics
2	Jackie Joyner-Kersee	UK	Track
3	Michael Phelps	USA	Swimming

Select Statement: Joins

- Find the names of all athletes who competed in Athens Olympics and their country

A SQL query walks up to two tables in a restaurant and asks: "Mind if I join you?"

```
SELECT A.name, A.country
FROM Athlete A, Competes C, Olympics O
WHERE A.aid = C.aid AND C.oid = O.oid
      AND O.year = 2004
```

How to Understand Joins

```
SELECT T1.A1, …, Tn.An
FROM T1, T2, …, Tn
WHERE condition
```

1. Cross product: Consider all combinations of each row from T1, with each row from T2, …., with each row from Tn (joined into a long row)
2. Filter: Discard rows that do not satisfy the WHERE clause condition
   - Example: if you have Ti.A = Tj.A in your conditions, then each row from Ti only joins with those rows in Tj that have the same value for attribute A

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c1	c2	d1

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c1	c2	d1
a1	b1	b1	c1	c3	d2

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c1	c2	d1
a1	b1	b1	c1	c3	d2
a1	b1	b1	c2	c2	d1
a1	b1	b1	c2	c3	d2

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c1	c2	d1
a1	b1	b1	c1	c3	d2
a1	b1	b1	c2	c2	d1
a1	b1	b1	c2	c3	d2
a1	b1	b2	c3	c2	d1
a1	b1	b2	c3	c3	d2

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
~~a1~~	~~b1~~	~~b1~~	~~c1~~ ≠	~~c2~~	~~d1~~
~~a1~~	~~b1~~	~~b1~~	~~c1~~ ≠	~~c3~~	~~d2~~
a1	b1	b1	c2	c2	d1
~~a1~~	~~b1~~	~~b1~~	~~c2~~ ≠	~~c3~~	~~d2~~
~~a1~~	~~b1~~ ≠	~~b2~~	~~c3~~ ≠	~~c2~~	~~d1~~
~~a1~~	~~b1~~ ≠	~~b2~~	~~c3~~	~~c3~~	~~d2~~
~~a2~~	~~b2~~ ≠	~~b1~~	~~c1~~ ≠	~~c2~~	~~d1~~
~~a2~~	~~b2~~ ≠	~~b1~~	~~c1~~ ≠	~~c3~~	~~d2~~
~~a2~~	~~b2~~ ≠	~~b1~~	~~c2~~	~~c2~~	~~d1~~
~~a2~~	~~b2~~ ≠	~~b1~~	~~c2~~ ≠	~~c3~~	~~d2~~
~~a2~~	~~b2~~	~~b2~~	~~c3~~ ≠	~~c2~~	~~d1~~
a2	b2	b2	c3	c3	d2

Example

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c2	c2	d1
a2	b2	b2	c3	c3	d2

A	C
a1	c2
a2	c3

Alternate Interpretation

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

When T.attr=T'.attr we don't need to consider all combinations, just those rows of T & T' that match on attr

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

Alternate Interpretation

```
SELECT T1.A, T2.C
FROM T1, T2, T3
WHERE T1.B = T2.B
  AND T2.C=T3.C
```

When T.attr=T'.attr we don't need to consider all combinations, just those rows of T & T' that match on attr

T1

A	B
a1	b1
a2	b2

T2

B	C
b1	c1
b1	c2
b2	c3

T3

C	D
c2	d1
c3	d2

A	B	B	C	C	D
a1	b1	b1	c2	c2	d1
a2	b2	b2	c3	c3	d2

A	C
a1	c2
a2	c3

Structured Query Language (SQL)

- Create a Table `Create`
- Add new records `Insert`
- Delete records `Delete`
- Update records `Update`
- Retrieve records `Select`

- Create a View `Create`
- Update a View `Update`

Views

- View is used just like a relation, but we store a definition, rather than a set of tuples.

```
CREATE  VIEW  Athens_Olympians (aid, name, country)
      AS SELECT A.aid, A.name, A.country
      FROM Athlete A, Competes C, Olympics O
      WHERE A.aid = C.aid AND C.oid = O.oid
            AND O.year = 2004
```

JOIN

- Provide external data independence
- Security
- Views can be dropped using `DROP VIEW`
- How to drop a table if there is a view on it?
  - `DROP TABLE` command has options to let user specify this

Updates on Views?

- User perspective: view is like a table
  - Fine for queries, but what about updates?

- Can be tricky to figure out how updates map back to data stored in base tables

Updates on Views: Example

Athlete

aid	name	age
1	Alice	18
2	Alice	25
3	Bob	22

Sport

sid	name	sport
1	Alice	tennis
2	Alice	frisbee
3	Bob	skating

```
CREATE VIEW ActiveStudents
AS SELECT A.name, A.age, S.sport
FROM Athlete A, Sport S
WHERE A.name = S.name
```

ActiveStudents

name	age	sport
Alice	18	tennis
Alice	18	frisbee
Alice	25	tennis
Alice	25	frisbee
Bob	22	skating

Updates on Views: Example

Athlete

aid	name	age
1	Alice	18
2	Alice	25
3	Bob	22

Sport

sid	name	sport
1	Alice	tennis
2	Alice	frisbee
3	Bob	skating

```
CREATE VIEW ActiveStudents
AS SELECT A.name, A.age, S.sport
FROM Athlete A, Sport S
WHERE A.name = S.name
```

ActiveStudents

name	age	sport
Alice	18	tennis
Alice	18	frisbee
Alice	25	tennis
Alice	25	frisbee
Bob	22	skating

What if we want to delete the row (Alice, 18, tennis) from ActiveStudents?

Updates on Views

- **Key Idea:** Guarantee that update can be mapped to precisely one tuple in one base table

- SQL-99
  - Can update field of a view if it is obtained from exactly one base table, and primary key of that table included in view

Relational Model: Summary

- A tabular representation of data
- Simple and intuitive
- Currently the most widely used database model
- Integrity constraints can be specified by the DBA, based on application semantics. DBMS checks for violations.
  - Two important ICs: **primary** and **foreign keys**
  - We **always** have domain constraints

    e.g. `INTEGER` fields must always contain integer values
- Views can be used for external schemas, and provide logical data independence

Students

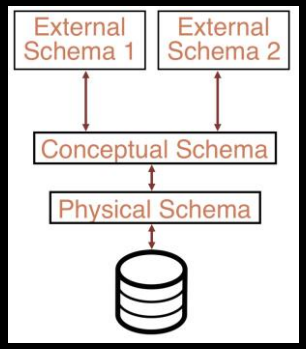
sid	name	login	age
13	Lisa	lsimp	40
41	Bart	bart	20

Courses

cid	cname	Cr.
E-484	EECS484	4
E-584	EECS584	3

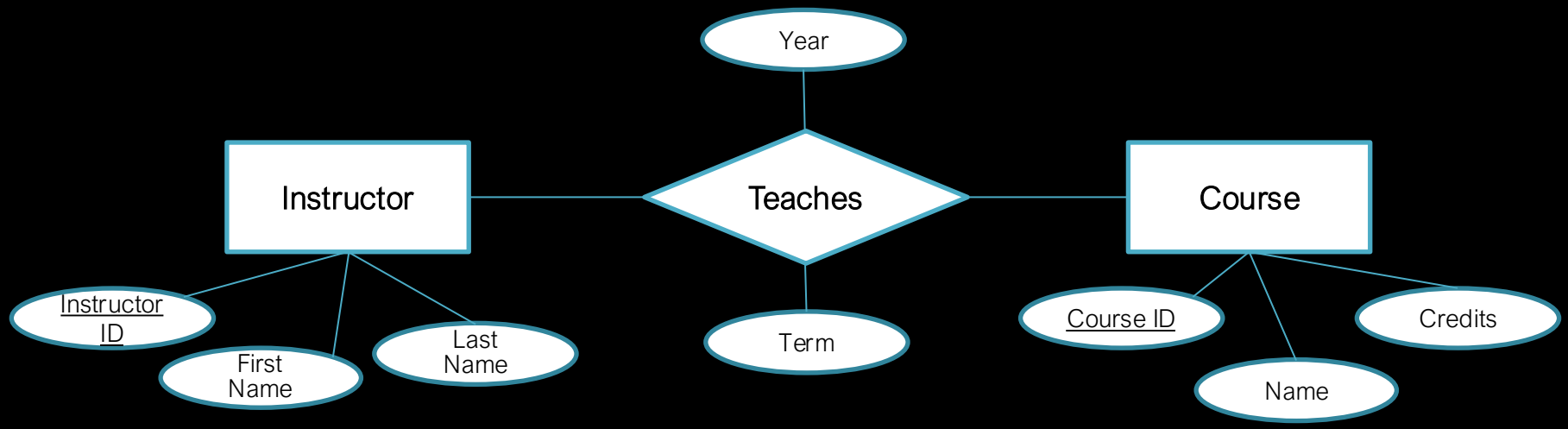
Enrolled

sid	cid	Grade
41	E-484	A-
13	E-584	A+



Next

- Discussion: Project 1 intro
- Tuesday: Translation from ER diagrams to Relations



Instructor ID	First Name	Last Name
394953	John	Smith
454544	Sara	King
439849	Alex	Dee
....	....	....
....	....	....

Instructor ID	Course ID	Year	Term
454544	E302	2009	F
394953	C210	2010	W
439849	M184	2010	F
....	....	....	....
....	....	....	....

Course ID	Name	Credits
M184	Calculus	3
C210	Physics	4
E302	Algorithms	4
....	....	....
....	....	....